

```
#!/usr/bin/env python3
"""
Protótipo: Núcleo (Global Workspace) +
Periféricos (módulos ativo-inferenciais)
=====
=====
Cada "periférico" é um módulo especialista
que:
    - observa um canal próprio (ruidoso) do
    ambiente
    - mantém uma crença ( $\mu$ ) sobre o estado
    real desse canal
    - atualiza a crença minimizando erro de
    predição ponderado por precisão
      (isso É o núcleo matemático do Free
    Energy Principle / Active Inference:
      update = ganho * precisão *
    erro_de_predição)
    - gera um sinal de "saliência" = precisão
    * erro2 (quanto mais surpreendido
      E mais confiante o módulo estava, mais
    ele "grita")

O "núcleo" (Global Workspace) a cada passo:
    - observa a saliência de todos os
    periféricos
    - se o máximo ultrapassa um limiar ->
    "ignição": o conteúdo do módulo
      vencedor é transmitido (broadcast) de
    volta a todos os outros módulos,
```

empurrando suas crenças na direção do vencedor (viés top-down)

Isso é uma implementação mínima, mas real, da combinação

Active Inference (periférico) + Global Workspace Theory (núcleo/hub).

Não simula consciência nem  $\Phi$  – simula o mecanismo formal de competição e broadcast que a literatura de GWT propõe como correlato funcional dela.

"""

```
import numpy as np
import matplotlib.pyplot as plt
```

```
np.random.seed(7)
```

```
#
```

```
=====
=====
```

```
# PARÂMETROS
```

```
#
```

```
=====
=====
```

```
K = 4                                # número de
módulos periféricos (ex: visão, audição,
                                # interocepção,
"linguagem interna")
```

```
labels = ['Visão', 'Audição',
'Interocepção', 'Linguagem interna']
```

```
T = 400                                # passos de tempo
dt = 1.0
```

```
kappa = 0.25                            # taxa de
```

```

aprendizado (atualização de crença)
precision_ema = 0.05      # taxa de
atualização da precisão (EMA do erro2)
ignition_threshold = 2.5  # limiar de
saliência para disparar broadcast
broadcast_gain = 0.5      # força do
empurrão top-down após ignição
obs_noise = 0.3           # ruído de
observação de cada canal
refractory_period = 15    # passos de
"descanso" de um módulo após transmitir
                        # (análogo
computacional de práticas que reduzem
ruminação)

#
=====
=====
# AMBIENTE: cada canal tem uma dinâmica
real distinta,
# com um "evento" de mudança abrupta em
tempos diferentes
# (simula algo relevante acontecendo em uma
modalidade)
#
=====
=====
def true_signal(k, t):
    base_freqs = [0.02, 0.035, 0.015, 0.05]
    event_times = [100, 180, 260, 320]
    # quando cada canal tem uma mudança abrupta
    val = np.sin(2 * np.pi * base_freqs[k]
    * t)
    if t > event_times[k]:

```

```

        val += 1.5    # salto de nível,
simulando evento saliente na modalidade k
        return val

#
=====
=====
# ESTADO INICIAL
#
=====
=====
mu = np.zeros(K)          # crença atual de
cada módulo
precision = np.ones(K) * 1.0
err_ema_sq = np.ones(K) * 1.0

history = {
    'mu': np.zeros((T, K)),
    'true': np.zeros((T, K)),
    'salience': np.zeros((T, K)),
    'precision': np.zeros((T, K)),
    'winner': np.full(T, -1),
    'ignition': np.zeros(T, dtype=bool),
}

#
=====
=====
# LOOP PRINCIPAL
#
=====
=====
refractory_until = np.zeros(K, dtype=int)
# passo até o qual cada módulo está

```

```

"descansando"

for t in range(T):
    true_vals = np.array([true_signal(k, t)
    for k in range(K)])
    obs = true_vals + np.random.randn(K) *
    obs_noise

    # --- Etapa periférica: erro de
    predição e atualização de crença (active
    inference) ---
    err = obs - mu
    salience = precision * err**2

    mu = mu + kappa * precision *
    err          # update de crença
    (gradiente de energia livre)
    err_ema_sq = (1 - precision_ema) *
    err_ema_sq + precision_ema * err**2
    precision = 1.0 / (0.1 +
    err_ema_sq)          # precisão
    inversamente prop. ao erro recente
    precision = np.clip(precision, 0.1,
    5.0)

    # --- Etapa do núcleo: competição e
    broadcast (Global Workspace) ---
    eligible = np.array([salience[k] if t
    >= refractory_until[k] else -1.0 for k in
    range(K)])
    winner = int(np.argmax(eligible))
    ignite = eligible[winner] >
    ignition_threshold
    if ignite:

```

```

        refractory_until[winner] = t +
refractory_period
        for j in range(K):
            if j != winner:
                mu[j] = mu[j] +
broadcast_gain * (mu[winner] - mu[j])

        # --- registro ---
        history['mu'][t] = mu
        history['true'][t] = true_vals
        history['salience'][t] = salience
        history['precision'][t] = precision
        history['winner'][t] = winner if ignite
else -1
        history['ignition'][t] = ignite

#
=====
=====
# ANÁLISE
#
=====
=====
n_ignitions = history['ignition'].sum()
winner_counts = {k: int((history['winner']
== k).sum())) for k in range(K)}

print("=" * 70)
print("RELATÓRIO: NÚCLEO (WORKSPACE) +
PERIFÉRICOS (ACTIVE INFERENCE)")
print("=" * 70)
print(f"Total de passos: {T}")
print(f"Total de eventos de ignição
(broadcast disparado): {n_ignitions}")

```

```

print("\nVitórias de broadcast por módulo
(quem 'tomou o workspace'):")
for k in range(K):
    print(f"  {labels[k]:20s}:
{winner_counts[k]:3d} vezes")

print("\nEventos de ignição no tempo (passo
-> módulo vencedor:")
ignition_steps =
np.where(history['ignition'])[0]
for t in ignition_steps[:20]:
    print(f"  t={t:3d} ->
{labels[int(history['winner'][t])]}")
if len(ignition_steps) > 20:
    print(f"  ... e mais
{len(ignition_steps)-20} eventos")

#
=====
=====
# PLOTS
#
=====
=====
fig, axes = plt.subplots(3, 1, figsize=(13,
11), sharex=True)

colors = ['tab:blue', 'tab:orange',
'tab:green', 'tab:red']

ax = axes[0]
for k in range(K):
    ax.plot(history['true'][:, k],
color=colors[k], alpha=0.4, linewidth=1,

```

```

label=f'{labels[k]} (real)')
    ax.plot(history['mu'][:, k],
            color=colors[k], linewidth=1.8,
            linestyle='--', label=f'{labels[k]}
(crença)')
ax.set_ylabel('Valor do sinal')
ax.set_title('Sinal real de cada canal vs.
crença do módulo periférico
correspondente')
ax.legend(fontsize=7, ncol=4, loc='upper
left')
ax.grid(True, alpha=0.3)

ax = axes[1]
for k in range(K):
    ax.plot(history['salience'][:, k],
            color=colors[k], linewidth=1.2,
            label=labels[k])
ax.axhline(y=ignition_threshold,
            color='black', linestyle=':', label='limiar
de ignição')
ax.set_ylabel('Saliência (precisão ×
erro²)')
ax.set_title('Competição por saliência
entre periféricos')
ax.legend(fontsize=7, ncol=5, loc='upper
left')
ax.grid(True, alpha=0.3)

ax = axes[2]
for t in ignition_steps:
    k = int(history['winner'][t])
    ax.scatter(t, k, color=colors[k], s=25)
ax.set_yticks(range(K))

```



```
ax.set_yticklabels(labels)
ax.set_xlabel('Tempo (passos)')
ax.set_ylabel('Módulo transmitido')
ax.set_title('Eventos de ignição/broadcast:
qual periférico "tomou" o workspace
global')
ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('/home/claude/active_inference_
workspace/workspace_demo.png', dpi=150)

print("\nArquivo gerado:
workspace_demo.png")
```